简历描述

Sourcing AI Agent

- 完整的工作流：
- 更详细的流程描述
- BuildProduct页面
- Tech Pack完整生命周期：
- 为什么要生成Teck Pack，这是行业标准吗？

AI Copilot

- 描述：有哪些功能？
- 难点1：如何实现AI聊天流式对话？

Pietra Landing Pages

- 重点：
- SSR（Server-Side Rendering，服务端渲染）
- ISR（Incremental Static Regeneration，增量静态再生成）
- 图片优化
- SEO优化
- 完整的 SEO 优化体系
- Bundle优化

pietra/auto-import 和 pietra-auto-tracker

- @pietra-auto-tracker
- @pietra/auto-import

Pietra Devtool

- 问题：怎么启动的一个专有的端口？
- 难点1: devtool页面如何和Vite服务器通信？
- 难点2：devtool页面如何和主项目进行通信？
- 怎么建立通信通道？

升级项目架构至 Monorepo 模式

- ❓ 1：为什么选择 Monorepo，而不是多个仓库？
- ❓ 2：按需加载是怎么做的？

主导团队将车控 SDK 重构为微应用框架

Sourcing AI Agent

描述：AI 自动帮用户分析采购需求、智能匹配全球工厂、一键批量发送 RFP，从 2–4 小时的手动工作缩短到 5 分钟。

完整的工作流：

用户在主页面输入需求

↓

Step 1: 创建 Tech Pack (保存采购规格)

↓

Step 2: AI 多轮对话 (补充细节、确认需求)

↓

Step 3: AI 自动匹配工厂 (后端算法)

↓

Step 4: 前端获取工厂详情 (调用 API)

↓

Step 5: 用户审核 (删除/编辑工厂)

↓

Step 6: 一键批量发送 RFP (邮件/WhatsApp)

↓

✓ 完成：工厂开始报价

更详细的流程描述

用户输入描述 （在输入时，可以上传产品的pdf文件，或者图片或者URL作为附带信息）

↓

点击按钮（handleGetStarted）

↓

【验证阶段】

├── 检查是否在加载中

├── 检查是否手机端

├── 检查输入内容是否为空

├── 检查描述长度（≥20字）

└── 检查用户余额

↓

【初始化阶段】(handleGenerateTechPack)

├── 清空状态数据

├── 保存用户输入

├── 设置Redux状态

└── 发送事件追踪

↓

【页面切换】

└── setToBuildPage(true) → 切换到BuildProduct组件

↓

【BuildProduct接管】

├── 自动生成产品描述

├── 初始化AI对话

└── 展示Tech Pack编辑界面

BuildProduct页面

前端】BuildProduct

↓

POST /ai-sourcing/generate-desc

{

doc: "https://cloud.s3/file.pdf",

url: "https://amazon.com/product/xyz",

product: "黑色棉布T恤，L码，轻薄舒适"

}

↓

【后端】AI分析

├── 提取关键特征（颜色、材质、尺码）

├── 生成产品描述建议

└── 返回轮询URI

↓

【前端】intervalResultForUri（长轮询）

轮询 https://api/polling/task-123

检查状态...

检查状态...

✅ 完成！返回list

↓

【前端】dispatch(setAgentTypeProductName('黑色棉T恤'))

一旦 `agentTypeProductName` 被设置，第二个 `useDebounceEffect` 立即触发AI聊天，主动发送第一条消息：

TypeScript

Tech Pack完整生命周期：

```
【BuildProduct 挂载】
    ↓
generateData = null
agentTypeProductName = ''
    ↓
【100ms 后】useDebounceEffect 触发
    ↓
handleGenerateDesc(initData)
    ↓
API: aiSourcingGenerateDesc({doc, url, product})
    ↓
轮询等待 AI 分析结果
    ↓
✅ agentTypeProductName = '黑色棉T恤'
    ↓
【10ms 后】第二个 useDebounceEffect 触发
    ↓
handleSendMessage(系统提示)
    ↓
sendAiChat(系统提示)
    ↓
API: aiSourcingChat({
  product: '黑色棉T恤',
  question: '系统提示',
  previousResponseId: undefined
})
    ↓
```

【后端AI Agent处理】

分析产品名称 → 生成产品规格 → 构建Tech Pack结构

↓

返回 productBrief 包含所有规格信息

↓

✅ setGenerateData({

productName: '黑色棉T恤',

packages: [{...}, {...}, {...}]

})

↓

【UI 更新】PlanOverview 显示完整的Tech Pack

用户可以编辑规格参数

为什么要生成Teck Pack，这是行业标准吗?

Teck Pack 是 产品制造业的实际工作工具——是连接设计者和生产工厂之间的"沟通文档"。

用户需求：

"我想源采一件黑色棉T恤"

❌ 直接发给工厂的问题：

工厂回复：什么款式？什么面料克重？什么印花工艺？

用户：我不知道...

工厂：我们无法报价

✅ 有Tech Pack后：

用户有一份完整的规格文档 → 发给工厂

工厂立即知道：

├── 材质：100% Cotton, 200 gsm

├── 尺码：XS-XXL

├── 工艺：Screen print

├── 颜色：Black, Navy, White

├── 年产量：5,000 pcs

└── 交货期：70天

AI对话框 MessageContent组件

难点1：不是和AI直接交互，是发送消息给后端，由后端处理后，再把消息一次性完整返回，需要前端手动处理打字机动画。

难点2：一开始进入该画面，Teck Pack还没生成，此时先默认等待5s开始打印欢迎动画，等欢迎文字动画打印完之后再开始打印AI消息文字动画，一开始他们不知道如何处理时序问题，之前的实现是进入该页面时，经常出现欢迎消息和AI消息都同时开始打印，不具备前后顺序性。我抽象出一个单独的hook，专门用于打字机动画，通过调用两次hook，保证单一职责，来保证欢迎消息和AI消息的时序性

难点3: 打字机动画的流畅性以及最佳实现。原来的hook是 通过 ref 获取 DOM 元素，每 speed 毫秒直接修改 element.textContent，从 DOM 中读取完整文本并逐字显示。固定速度，一个一个蹦，缺乏AI感，直接调用DOM，性能有问题。我实现的hook完美解决了这些缺点，完全由数据驱动，只变更state的状态，不直接更改dom，单一职责，更好控制打字机的启动和停止，而且还可以手动调整打字机的打字速度以及根据随机数生成消打字机动画，非常贴合真实AI聊天。

AI Copilot

描述：有哪些功能?

1. 提问：用户输入问题，AI 给出回答
2. 上传文件：用户可上传图片/文档作为 AI 参考
3. 快速模板：直接点击推荐卡片触发预设问题对话

1. 基础聊天功能

- 用户输入文本提问，AI 通过 SSE（Server-Sent Events）流式提供实时响应
- 支持多轮对话，消息自动滚动到底部
- 清空聊天、重试失败的请求

2. 文件上传与附件

- 用户可上传图片、PDF、Word、CSV 等文件
- 上传的文件被索引并作为附件发送给 AI
- 可删除已上传的文件
- 显示附件列表

3. AI 思考过程展示

- 显示 AI 的 "thinking" 消息（思考步骤）
- 显示 AI 调用的工具名称（tool_text）
- 动画展示思考过程

4. 多种响应内容类型

- 纯文本：普通消息
- 生成 HTML：AI 可生成交互式 HTML 页面
- 生成 Markdown：格式化文本
- 生成图片：显示 AI 生成的图片列表
- 任务详情（taskDetail）

- 中断消息（interrupt）：需要用户确认的操作

5. 对话历史管理

- 查看之前的聊天记录（Previous chats）
- 加载对话的全部历史消息（支持分页加载）
- 点击历史对话直接打开

难点1：如何实现AI聊天流式对话?

流式对话实现流程：

1. 用户输入 → 点击发送
2. 创建 SSE (Server-Sent Events) 连接到 `/agent/sse-ask-ai?conversationId=...`
3. 后端开始流式返回 JSON 事件（每行一个）
4. 前端监听 `onmessage` 事件
5. 解析 JSON，根据 `type` 字段分类处理（thinking，tool_text，tool_result，text）根据类型分别进行渲染
6. 动态追加文本到消息对象
7. 每次有新数据就 `setMessages()` 触发重新渲染
8. UI 显示逐字输出效果
9. 收到 `[DONE]` 标记，断开连接
10. 保存对话摘要到数据库

Pietra Landing Pages

重点：

- NextJS SSR + ISR
- 图片优化：Next.js Image 组件自动优化
- SEO：自动生成 Sitemap，支持 Google Tag Manager
- 部署：支持 S3 缓存、CDN（CloudFront）

SSR（Server-Side Rendering，服务端渲染）

工作原理：

- 每次用户请求页面时，服务器实时生成 HTML
- **服务器执行 React 组件和数据获取（`getServerSideProps`）**
- 返回完整的 HTML 给浏览器

优点：
✅ 数据总是最新的
✅ 适合频繁变化的内容（用户信息、实时数据）
✅ 动态内容完全支持

缺点：

❌ 每次请求都需要计算，响应慢
❌ 服务器压力大
❌ 高流量时容易成为瓶颈
❌ 用户体验差（等待时间长）

ISR（Incremental Static Regeneration，增量静态再生成）

工作原理：

在构建时预先生成页面（生成静态 HTML）

用户访问时立即返回静态 HTML（极快）

后台定时重新生成页面（`revalidate`）

重新生成后的新页面替换旧页面

优点：

✅ 响应极快（返回静态 HTML）
✅ 服务器压力小
✅ 支持大规模流量
✅ SEO 友好（静态 HTML）
✅ 用户体验好
✅ 可以手动按需重新生成（revalidate）

缺点：

❌ 不能实时更新（有延迟）
❌ 内容更新需要等待重新生成
❌ 不适合用户特定的动态内容

使用场景：

博客文章（更新不频繁）

产品列表（几分钟或几小时更新一次）

营销页面（大流量，内容变化慢）

文档、FAQ

之前所有页面都是SSR，SEO效果极差，后来采用混合渲染策略：

首先，我们有 6 个 SSR 页面，主要是需要实时数据的页面，

比如职位招聘页和 AI 工具，这些页面数据变化频繁，无法提前生成。

其次，我们有 8 个 ISR 页面，特别是博客系统，这些内容由 Prismic CMS 管理，

更新周期较长，我们使用 getStaticProps + getStaticPaths，

让首次访问时会生成页面，之后的访问返回缓存的静态 HTML。

这样做既保证了性能（TTFB < 50ms），又能定期更新内容。

最后，其他页面保持完全静态，进一步减少服务器压力。

从 SEO 角度看，这个策略很有效：

– ISR 和 Static 页面天然提供完整 HTML，搜索引擎爬虫友好

– 我们正确使用 canonical 标签避免重复内容

– 页面速度快，Core Web Vitals 评分高

– 结构化数据也能被完美爬取

这个组合既优化了成本，又优化了用户体验和 SEO。"

图片优化

我们实施了CDN 边缘计算 + 响应式加载 + 格式自动优化的三层图片优化方案。CloudFront 加载器根据请求设备尺寸和浏览器能力自动选择最优格式和大小，减少约 50% 的图片传输。同时，通过 sizes 属性实现真正的响应式图片，而不是简单的缩放。这套系统使得博客等高图片占比的页面 LCP（最大内容绘制）时间从 3.5s 降低到 1.2s。"

1. 自定义图片加载器：

```
TypeScript
```

- ✅ 利用 CloudFront 进行边缘计算，在全球 CDN 节点实时处理图片
- ✅ 自动选择最优格式（WebP、AVIF、JPG 等），兼容旧浏览器
- ✅ 根据设备尺寸动态调整图片尺寸，减少传输数据量

2. Nextjs Image组件优化

```
JavaScript
```

- 生产环境使用自定义 CloudFront 加载器
- 开发环境使用 Next.js 默认加载器（快速反馈）
- 支持多个图片来源（S3、Prismic CMS、Shopify）

3. 响应式图片实现

```
JavaScript
```

📱 手机（<832px）：返回 `calc(100vw - 32px)` 宽度的图片

🖥️ 桌面（≥832px）：返回 800px 宽度的图片

自动生成 srcset，浏览器自动选择最合适的版本

4. 懒加载和预加载

```
JavaScript
```

5. SVG 特殊处理

```
JavaScript
```

- SVG 作为 React 组件使用，支持动态修改

- 原生 SVG 保持矢量特性，无损放大
- 缓存友好

6. 缓存优化

Nextjs缓存策略：

- 公有文件(/public) → Cache-Control: max-age=0
- 图片优化结果 → Cache-Control: max-age=31536000 (1年)

SEO优化

1. SSR + ISR 提升搜索引擎抓取效率
2. 图片优化，提升LCP、FCP
3. Sitemp自动生成，提交到Google Seerach Console

自动为所有页面生成多个 XML Sitemap

- 按分类分离（博客、产品、供应商），便于搜索引擎爬虫高效索引
- 支持 100+ 个产品分页 Sitemap（避免单个文件过大）
- 每周自动更新（changefreq: weekly）

4. 为每个页面增加结构化数据（JSON-LD）

- 帮助搜索引擎理解页面内容
- 在搜索结果中显示星级评分、组织信息
- 提升 Rich Snippet 竞争力

5. Meta标签优化

- 标题和描述精确控制（影响搜索结果点击率 CTR）
- 社交分享优化（OG 标签控制分享时的卡片样式）
- Canonical 避免重复内容被扣分
- Google 验证证明站点所有权

6. Lighthouse 性能监控

   监控指标：

- Lighthouse Performance Score（90+ 是优秀）
- Core Web Vitals（LCP、FID、CLS）
- SEO Score
- Accessibility Score

  作用：

- 持续监控页面性能
- 性能下降时自动告警
- Google 搜索排名直接受 Core Web Vitals 影响

完整的 SEO 优化体系

优化项	实现方式	搜索引擎收益
Sitemap	自动生成 100+ 个分类 Sitemap	快速索引，覆盖全面
Robots.txt	指导爬虫，避免参数重复	提高爬虫效率 40%
JSON-LD	34 个页面带结构化数据	出现 Rich Snippet
元标签	og:, twitter:, canonical	社交分享优化、避免重复
HTML 优化	lang, preconnect, dns-prefetch	Core Web Vitals 优化
性能监控	Lighthouse CI 自动化	持续保持排名优势
速度优化	结合图片优化 + ISR	页面速度提升 50%

Bundle优化

我们项目初期的生产 Bundle 是 2.8MB，首页加载 5+ 秒。我通过一套系统的优化策略将其减少到 920KB，降幅 67%。

核心优化包括三个方面：

首先是代码分割。利用 Next.js 的 dynamic() 动态导入，把 交互密集的页面单独打包，只在用户访问时加载。这减少了 45% 的额外 JS。

其次是 Tree-shaking。确保 package.json 中设置 sideEffects: false，这样 webpack 能安全地移除未使用的代码，减少约 12%。

第三是包结构优化。通过 splitChunks 将 React、Ant Design 等库分离到独立的 vendor chunk，这些文件被浏览器缓存，用户回访时只需加载新代码。

结果是首屏 LCP 从 5.2 秒降到 1.8 秒，用户停留时间增加 23%，有机搜索排名前 3 的关键词增加了 182% — 从 45 个到 127 个。

我们通过 Bundle Analyzer 和 Lighthouse CI 持续监控，确保没有性能回退。

追问	你的回答
怎么衡量优化成果的?	"用 @next/bundle-analyzer 生成交互式报告，对比优化前后的 chunk 大小。同时用 Lighthouse 测量 Core Web Vitals。"
有没有遇到什么坑?	"SSR: false 的页面首屏没有内容，需要用骨架屏占位。还有动态导入增加了首页的 JS 解析时间，但总体来说 3-5ms 的成本换来 65% 的加载速度提升，值得。"
怎么避免优化后又变大?	"几个策略：CI 门槛检查、定期 bundle 分析、代码审查检查清单、依赖更新时对比大小。"

如果团队有 20 个页面，怎么一个个优化?	"不用一个个优化。核心优化（vendor 分离、tree-shaking）对整个项目生效。然后在大页面（>500KB）上加动态导入。优先级是：博客（高流量）> 产品页 > 其他。"

问：Webpack 的 tree-shaking 是怎样工作的?

答："Tree-shaking 依赖 ES2015 的静态导入和导出。Webpack 在构建时分析整个模块图，标记所有未使用的导出为 'unused'。然后 Terser 压缩器在 minify 阶段完全移除这些代码。

核心前提是代码必须是 ESM（ES Modules）格式，因为 Webpack 需要静态分析才能知道哪些代码被使用。

实际上，我们还做了另一个优化：从 lodash 改用 lodash-es，因为后者是原生 ESM，能被 tree-shaking。这额外减少了 50KB。"

pietra/auto-import 和 pietra-auto-tracker

@pietra-auto-tracker

这是一个基于 Vite 和 Babel AST 的自动埋点插件，可以在编译阶段为 TSX/JSX 组件自动注入点击事件上报逻辑，减少前端手写埋点代码。这个插件让业务方只需要在 JSX/DOM 上声明式添加埋点字段，构建时自动注入上报逻辑，从而减少手写埋点代码，对业务 JavaScript 逻辑的侵入更小。

实现原理：

“这个插件本质上是一个编译期的代码转换工具。我把它接在 Vite 的 transform 钩子里，让它在构建时拿到每个 .tsx 文件的源码。拿到源码后，先用 @babel/parser 按 module 模式去解析，并开启 jsx 和 typescript 插件，这样 TSX 代码就会被转换成 Babel AST。然后我用 @babel/traverse 去遍历整棵 AST，重点关注两类节点：一种是 ImportDeclaration，用来检查当前文件里有没有从 @apis 引入埋点上报函数 sendMoreCommonEvent；另一种是 JSXOpeningElement，也就是 JSX 标签的开始节点，因为我要在这里识别业务方声明的 data-log-* 埋点属性。

当遍历到 JSXOpeningElement 时，我会先判断它身上有没有 data-log-common-type，这个字段相当于埋点开关；如果有，就继续把同一节点上的其他埋点字段，比如 data-log-field1、data-log-common-content 等提取出来，组装成一个 reportParams 对象。这里因为 JSX 属性的值可能是字符串字面量，也可能是表达式，比如变量、成员访问、函数返回值，所以我又做了一层节点转换，保证这些值在生成新代码时还能保持原来的表达式语义。

接下来就是注入逻辑。我会先构造几段新的 AST 节点，比如 const eventType = ...、const reportParams = ...，以及 sendMoreCommonEvent(eventType, reportParams) 这个调用表达式。然后根据原组件有没有 onClick 分情况处理：如果没有，我就直接创建一个新的箭头函数作为 onClick；如果原来 onClick 是函数表达式或者箭头函数，我就把埋点相关的变量声明插到函数体前面，再把上报调用插到函数体后面；如果 onClick 是一个函数引用，比如 handleClick，我就把它包装成一个新的箭头函数，里面先调用原函数，再执行埋点上报。这样既保留了原有业务行为，也把埋点逻辑自动织入进去了。

最后，如果前面遍历 import 时发现当前文件还没有引入 sendMoreCommonEvent，我就再用 @babel/types 构造一个 import { sendMoreCommonEvent } from '@apis' 的 AST 节点，插到程序最前面。全部 AST 修改完成后，再用 @babel/generator 把修改后的 AST 重新生成为字符串代码，返回给 Vite。这样最终打包出来的代码里就已经包含埋点逻辑了，业务侧只需要写声明式属性，不需要手动写上报代码。

这个实现的关键点有两个：第一，它是编译期注入，不影响运行时组件设计；第二，它是基于 AST 做结构化改写，不是简单字符串替换，所以可以比较稳定地处理不同形式的 onClick 写法。”

@pietra/auto-import

这个项目是一个给 Pietra 前端项目用的“自动导入插件”。它本身不是业务插件，而是一个开发期工具：在你写 React/Vite 代码时，允许你直接使用 useState、Button、map、cN、项目里的组件名、Pietra 图标名，而不用手写对应的 import 语句。平时写 React 项目时，你经常要在文件顶部手动写这些：

```
import { useState } from'react'
import { Button } from'antd'
import { map } from'lodash'
import cN from'classnames'
```

这个插件的作用就是：你不用手写这些 import，直接在代码里用 useState、Button、map、cN 就行，它会在构建时自动帮你补上。

对 Pietra 项目来说，它主要解决三类自动导入：

- 常用第三方库：比如 react、react-router、antd、lodash、classnames
- 项目内部组件：比如 src/components 下面的组件
- Pietra 图标：比如 @pietra/icons 里的图标

实现原理：

“这个插件的核心原理，是在 Vite 的编译流程里拿到源码，然后基于 AST 分析当前文件里用了哪些标识符，比如 useState、Button、map。如果发现这些变量被使用了，但源码里没有对应的 import，插件就会按照预先配置好的映射关系，自动往文件里补上对应的导入语句，比如把 useState 对应到 react，把 Button 对应到 antd。最后再把修改后的代码交还给 Vite 继续构建。”

Pietra Devtool

描述： DevTools 风格的应用内调试面板，重点在于把 CSS 变量、组件、Hook、状态、资源和结构关系做成可视化界面，帮助开发者更快理解和排查前端项目。类似Vue Tool工具

左侧菜单里有 Assets、Components、Hooks、Store、Graph、Inspect，说明它不只是看颜色，还覆盖了：

- 静态资源查看
- 颜色变量查看
- React 组件结构查看
- Hook 使用情况查看
- 状态管理数据查看

- 依赖/关系图可视化
- 元素检查与调试

问题：怎么启动的一个专有的端口?

本质上就是“借用 Vite 自己已经启动好的本地开发服务器，顺手再开一个调试页面入口”。

你平时启动项目时，Vite 会起一个本地服务，比如 localhost:5174。这个插件并没有再额外起一个新服务，而是在 Vite 启动的时候插进去一段中间件逻辑。这个中间件的作用是：如果浏览器访问的是 /__react_devtool__/ 这个地址，就不要走正常的业务页面了，而是返回一套 devtools 自己的前端页面。

所以你可以把它想成：

- 原来的项目首页是 localhost:5174/
- 插件额外挂了一个页面入口 localhost:5174/__react_devtool__/
- 两个页面共用同一个 Vite 服务、同一个端口
- 只是访问的路径不同，返回的内容不同

如果面试官问“原理是什么”，你可以回答：

Vite 开发服务器底层本来就是一个 Node 服务，它支持插件在启动时往服务器里注册中间件。这个项目就是利用这个能力，在服务器里加了一个专门处理 /__react_devtool__/ 的中间件。当请求打到这个路径时，中间件就把 devtools 对应的静态资源返回出去，所以浏览器就能打开这套调试界面。

难点1: devtool页面如何和Vite服务器通信?

核心思路
devtool 页面不是自己另开一套通信系统，而是直接复用了 Vite 开发服务器本来就有的 WebSocket 通道。

也就是说：

- 浏览器里打开 devtool 页面
- 这个页面会连回当前 Vite dev server 的 WebSocket
- 然后双方约定一个“专用消息标识”
- 后面 devtool 和 Vite 服务器之间，就通过这个通道互相发消息

这样就实现了通信。

为什么能这样做
因为 Vite 在开发模式下本身就有一条 WebSocket 连接，主要是给 HMR 热更新用的。这个项目等于是“借了这条现成的线”，不需要自己再开一个新的 socket 服务。

具体怎么做
你可以分成前后两端讲：

1. 前端 devtool 页面这边
   页面启动后，会先拿到 Vite 的 hot context，也就是 Vite 客户端通信对象。

它做的事情很简单：

- 先根据当前地址，算出 Vite 服务器地址

- 再通过 vite-hot-client 连上 Vite
- 然后创建一个 hot context，后面就能收发消息

1. Vite 服务端这边
   插件在 configureServer 里拿到 Vite server 实例，然后把这个 server 保存起来。

后面服务端会从 viteServer.hot 或 viteServer.ws 里拿到底层的 WebSocket 能力，用来监听和发送消息。

1. 双方怎么识别彼此的消息
   因为 Vite 的 WebSocket 本来还要处理热更新消息，所以 devtool 不能乱发，必须做消息隔离。

这个项目的做法是：
双方约定一个专门的事件 key，只处理这个 key 对应的消息。

你可以把它理解成：
同一个群聊里很多人在说话，但 devtool 只看“带某个暗号”的消息。

1. 为什么调用函数一样自然
   项目里用了 birpc。
   它的作用是把“发消息/收消息”包装成“像本地调用函数一样”的形式。

比如前端想拿资源列表，不是手写 websocket 消息，而是类似这样：

- 前端调用一个 RPC 方法
- RPC 把它序列化成消息
- 通过 Vite 的 WebSocket 发给服务端
- 服务端收到后执行对应函数
- 再把结果返回前端

所以开发体验上很像：
“前端直接在调服务端方法”。

一句话总结
这个项目里，devtool 页面和 Vite 服务器通信的关键，就是复用 Vite 自带的 WebSocket，再用 RPC 和自定义事件 key，把它包装成一套专门给 devtool 用的双向通信机制。

面试时可以直接说的版本
devtool 页面和 Vite 服务器通信，不是自己重新建了一套 socket，而是复用了 Vite 开发模式下已有的 WebSocket 连接。前端拿到 Vite 的 hot context，服务端拿到 Vite server 的 ws/hot 对象，双方通过一个约定好的事件 key 做消息隔离，再用 birpc 把底层消息封装成函数调用，这样 devtool 页面就能像调用本地方法一样去拿服务端的数据。

难点2：devtool页面如何和主项目进行通信?

先说结论
devtool 页面和主项目通信，不是直接去读主项目代码，也不是靠普通接口轮询，而是通过“注入到主项目里的一段桥接脚本 + 浏览器窗口之间的消息机制 + RPC”来完成的。

你可以把它理解成：
devtool 页面是一个单独的调试界面，主项目是被调试对象，这两边中间要有一个“桥”。

为什么需要这层桥
因为 devtool 页面虽然和主项目跑在同一个 Vite 服务上，但它们本质上还是两个不同页面、两个不同运

行上下文。

也就是说：

- 主项目页面里有真实的 React 组件树、状态、DOM、CSS 变量
- devtool 页面只是一个调试面板，它默认拿不到这些运行时数据

所以必须有一个桥，把主项目里的信息传给 devtool 页面。

这个项目怎么做
你可以拆成 3 步讲：

1. 先把桥接脚本注入到主项目页面
   在 Vite 插件里，通过 transformIndexHtml 给业务页面额外插入了一段脚本。

这段逻辑相当于：
当主项目首页 HTML 返回给浏览器之前，插件偷偷往里面塞了一段 script。

这样主项目页面一打开，这段脚本就会先执行。

1. 这段脚本在主项目里建立通信能力
   被注入的脚本是 initRpc.js，它运行在主项目页面里，作用就是把主项目和 devtool 页面连起来。

你可以理解为它做了两件事：

- 在主项目页面里拿到一套通信对象
- 把主项目里的运行时信息通过这套通信对象暴露出去

所以后面 devtool 页面就可以请求主项目数据，比如组件、状态、颜色变量这些。

1. devtool 页面再通过 RPC 调主项目暴露的方法
   最终效果不是“devtool 直接操作主项目”，而是：
   - 主项目先暴露一些能力
   - devtool 页面通过 RPC 去调用这些能力
   - 主项目执行后，把结果回传给 devtool

所以整个关系更像：

- Vite server 负责搭桥和转发
- 主项目负责提供真实运行时数据
- devtool 页面负责展示和调试

面试里建议这样总结
devtool 页面和主项目通信的关键，不是它们天然互通，而是插件在主项目 HTML 里注入了一段桥接脚本。这个脚本运行在主项目上下文里，能够拿到主项目的真实运行时数据，再通过浏览器侧通信机制和 RPC，把这些数据传给 devtool 页面。所以 devtool 才能看到组件、状态、样式变量这些原本只存在于主项目里的信息。

怎么建立通信通道?

一句话先说清
这个项目其实建了两条通信通道：

- devtool 页面 <-> Vite 服务器

- devtool 页面 <-> 主项目页面

第一条用的是 Vite 自带的 WebSocket
第二条用的是 浏览器 BroadcastChannel

1. devtool 页面和 Vite 服务器怎么建通道

这条通道是复用 Vite 本来就有的 HMR WebSocket。

建立过程可以理解成 3 步：

1. devtool 页面启动后，先拿到 Vite 的 hot context
   代码在 packages/client/src/main.tsx (line 10) 和 packages/client/src/initViteClient.ts (line 4)
2. 然后调用 createViteClientRpc()，把这条 Vite WebSocket 包装成 RPC 通道
   对应 packages/core/src/vite.ts (line 84)
3. 服务端这边，在插件的 configureServer 里拿到 server，再调用 createViteServerRpc()，两边就接上了
   对应 packages/vite/src/index.ts (line 64)

底层真正发消息的地方在 packages/core/src/channel.ts (line 9)：

- 客户端用 client.send(...)
- 服务端用 ws.send(...)
- 双方用同一个事件 key 识别消息

也就是说，本质上就是：
“借 Vite 的 WebSocket，当成 devtool 和服务端之间的专用电话线”。

2. devtool 页面和主项目页面怎么建通道

这条通道不是用 Vite WebSocket，而是用浏览器原生的 BroadcastChannel。

建立过程也很直接：

1. Vite 插件通过 transformIndexHtml 往主项目页面注入一个脚本 initRpc.js
   对应 packages/vite/src/index.ts (line 98)
2. 这个脚本运行在主项目页面里，调用 createBroadcastServerRpc(...)
   对应 packages/vite/src/initRpc.ts (line 164)
3. devtool 页面如果发现自己是独立窗口，就调用 createBroadcastClientRpc()
   对应 packages/client/src/main.tsx (line 16)
4. 双方底层都连到同一个 BroadcastChannel 名字上
   对应 packages/core/src/channel.ts (line 33)

这里的关键点是：

- 主项目页面和 devtool 页面虽然是两个页面
- 但它们在同一个浏览器环境里
- 所以可以通过 BroadcastChannel 互相发消息

你可以把它理解成：
“两个浏览器标签页加入了同一个聊天室，只要频道名一样，就能互相说话”。

3. 为什么看起来像直接调函数

因为项目在通道上面又套了一层 birpc。

所以真实过程是：

- 你在 devtool 页面里调用 getPrimaryRendererID()
- RPC 帮你把调用变成消息
- 通过 WebSocket 或 BroadcastChannel 发出去
- 对方执行对应函数
- 再把结果回传回来

所以开发体验上像“直接调对方方法”，但底层其实还是消息通信。

面试版简答
这个项目的通信通道分两层。devtool 页面和 Vite 服务器之间，复用了 Vite 自带的 WebSocket；devtool 页面和主项目页面之间，使用浏览器原生的 BroadcastChannel。然后再在这两个通道上套一层 RPC，把消息收发包装成函数调用，所以最终看起来像是页面之间可以直接互调方法。

升级项目架构至 Monorepo 模式

我们这个项目一开始是一个比较典型的 Vue2 单体应用，所有功能都堆在一个仓库里，组件、业务代码耦合比较严重。
随着业务不断扩展，逐渐出现几个问题：

- 代码结构混乱，不同业务之间相互影响
- 组件复用率低，很多逻辑重复实现
- 首屏加载体积越来越大，性能开始下降

所以后来我主导做了一次架构升级，主要分三步：

第一步是做 Monorepo 改造，把原来一个大仓库，拆成多个子模块，比如基础组件、业务组件、工具库等，统一在一个仓库里管理，同时规范依赖关系，这样代码边界会更清晰。

第二步是推进组件化设计，我把通用能力抽成基础组件库，比如 UI 组件、通用组件，以及一些业务组件，做到业务逻辑和通用能力解耦，提升复用性。

第三步是做按需加载优化，基于路由和模块维度做动态加载，把原来一次性加载的代码拆分成多个 chunk，减少首屏加载体积。

这次改造之后，一方面代码结构清晰了很多，团队协作成本下降；另一方面首屏性能也有明显提升。

这次改造其实不仅是技术调整，更偏向工程体系优化，我在里面主要负责方案设计和推进落地，包括模块划分、组件抽象边界，以及后续规范制定。

❓1：为什么选择 Monorepo，而不是多个仓库？

你可以答：

主要考虑到几个点：

- 各个模块之间是强关联的，比如组件库和业务模块
- 需要频繁联调，如果拆成多仓库会增加沟通和发布成本
- Monorepo 更适合统一依赖管理和版本控制

所以我们选择用 Monorepo 来做统一管理。

❓ 2：按需加载是怎么做的？

你可以补一句：

主要是基于 Vue Router 的动态 import，比如路由组件做懒加载，同时对一些大模块做拆分，避免打包成一个大 bundle。

主导团队将车控 SDK 重构为微应用框架

在改造之前，每次业务更新都必须走应用商店审核流程，周期比较长，也影响业务迭代效率。

所以我们通过引入微应用架构，把业务模块从主 App 中解耦出来，做成可以独立发布的微应用。

这样后续业务更新只需要在后台发布新的微应用包，用户在 App 内即可完成更新，不再依赖 App 上架流程。

1. 对于旧业务，我们搞了一个基座，用来打包每个微应用，其原理就是只导入某个具体业务模块的路由，然后进行打包，得到的包就是这个微应用的包，写了一个单独的cli进行配置，提供一个配置文件，里面声明要打包的模块，也就是包名，在终端执行打包命令之后，会根据配置的包名对每个微应用进行逐个打包，最终会得到所需微应用的包。 2. 对于新微应用，统一使用团队自研的脚手架进行项目初始化，项目初始化时，会自动注入和原生交互的SDK API，开发者可以通过API平台去查看每个API的使用方法，无需自己单独注入，也无需和原生进行对接开发，原生只开发原生，前端只开发前端，开发完成之后，每个业务组自己进行打包。 3. 使用压缩工具对每个微应用包进行压缩，对每个微应用的包进行命名，包括包名、版本号、SDK信息等 3. 将微应用的包上传至微应用管理平台，对每个微应用进行生命周期管理，包括历史版本，上架下架，类似于应用商店 4. 用户打开主App时，App会在后台自动检测微应用更新，在模块图标标亮提示用户可以升级该微应用，用户点击之后，微应用即可进行远程更新，其原理是，通过后端去拉取最新版本的包，然后下载到本地，再打开进入微应用。本质上我们是构建了一套基于资源包分发的动态化架构，让业务可以脱离宿主应用生命周期独立演进。

另外，长安汽车有很多个子品牌，他们都有各自独立的App，但是每个子品牌的车型或多或少都有各自不同或者相同的微应用，以前的开发范式是每个子品牌的App都有自己单独的开发人员，但是实际上有很多功能都是相同的，现在有了微应用，公司就可以将资源整合起来，一个模块一个业务团队，一次开发即可集成到不同的宿主App，不同品牌的App想要哪些微应用，他也可以自己动态组合，实现“一次开发，多端复用”。

这个框架直接推动了公司客户端研发人员的资源配置，极大提升了App的开发效率。最终这种方案不仅实施到手机App，还被采纳到车机App，惠及千万用户，为此还申请了两个技术发明专利。我当年也因为这个项目，被提升了我们组的技术leader，还代表长安汽车去参加了当年的重庆前端技术分享会。